

FIAP - Faculdade de Informática e Administração Paulista

Pós-Graduação em Arquitetura e Desenvolvimento Java

Jefferson Ricardo dos Santos

RM371825

John Pablo da Silva Gomes

RM372387

**SISTEMA DE GESTÃO DE RESTAURANTES: API RESTFUL COM CLEAN
ARCHITECTURE**

Tech Challenge — Fase 2

São Paulo

2026

DESCRIÇÃO DO PROJETO

Sistema de gestão de restaurantes desenvolvido com Spring Boot, seguindo os princípios da Clean Architecture. A aplicação permite o cadastro de tipos de usuário, usuários (donos de restaurante e clientes), restaurantes e itens do cardápio, com regras de negócio que garantem que apenas proprietários do tipo RESTAURANT_OWNER possam criar e gerenciar restaurantes e seus cardápios.

Repositório: <https://github.com/jeffesa/12ADJT-fase-2>

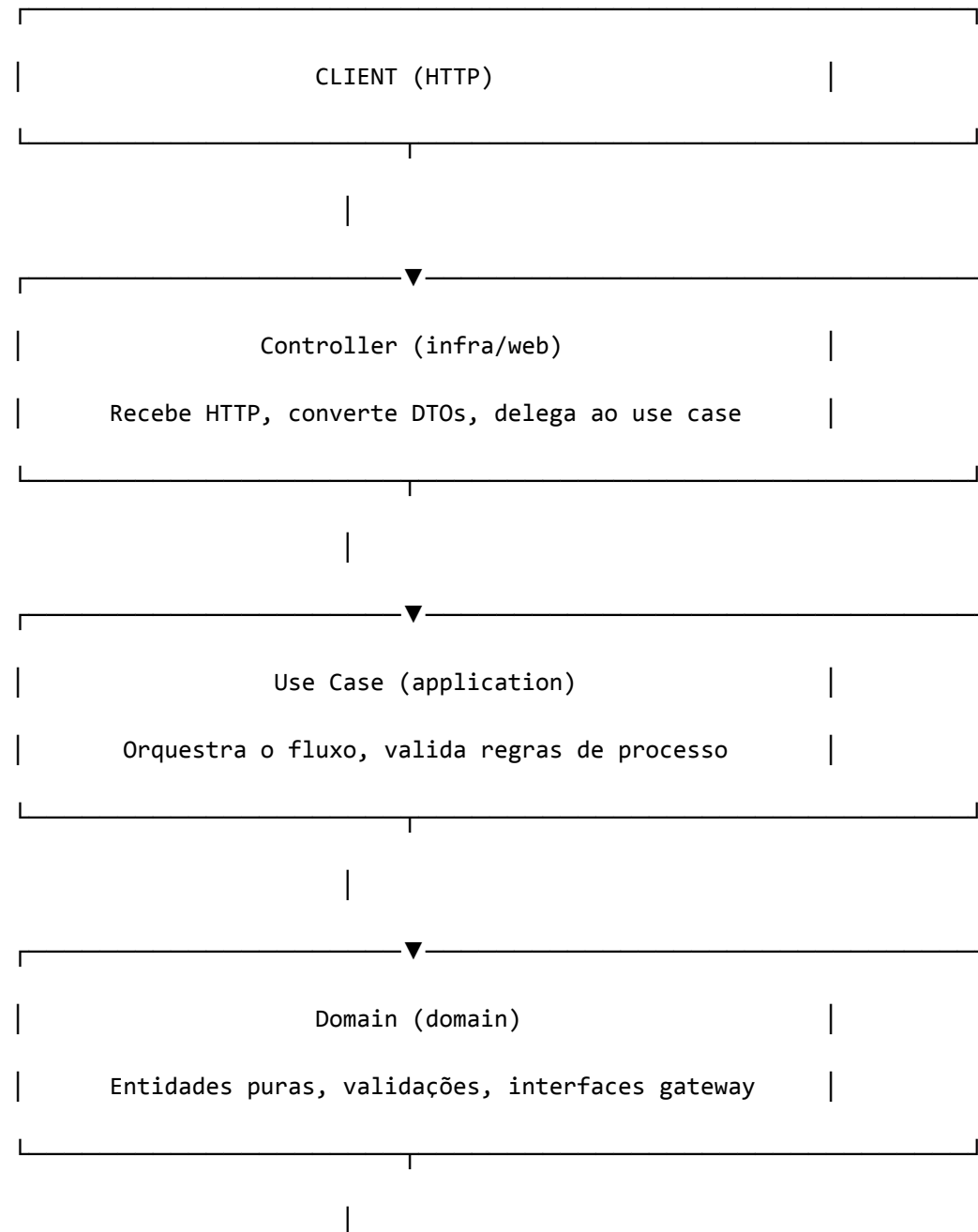
Aplicação em produção: <https://one2adjt-fase-2.onrender.com>

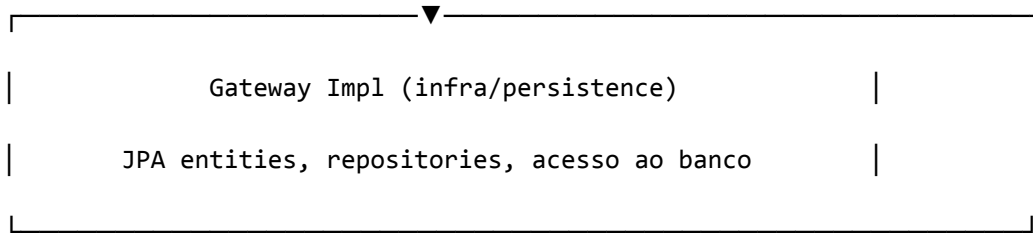
Vídeo de apresentação: (inserir link do YouTube)

ARQUITETURA

Clean Architecture

O projeto segue a Clean Architecture (Robert C. Martin), organizada em três camadas com dependência de fora para dentro:



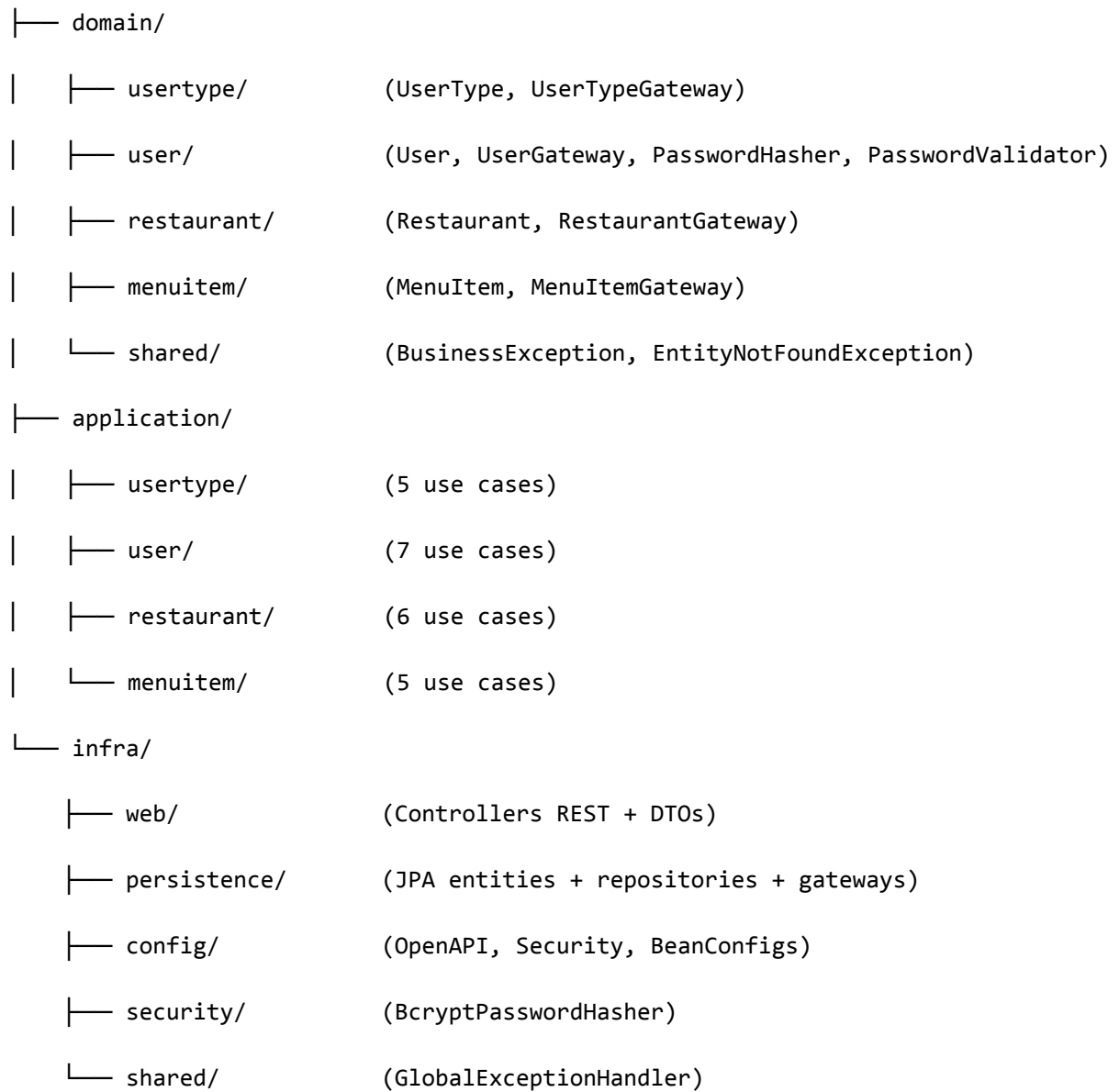


Regra de Dependência

- **domain/** → Não depende de nada. Contém entidades puras (POJOs), interfaces de gateway e exceções de domínio. Zero imports de framework.
- **application/** → Depende apenas do domain. Contém use cases que orquestram fluxos de negócio. Zero imports de framework.
- **infra/** → Depende de application e domain. Contém controllers REST, DTOs, entidades JPA, repositories e configurações Spring.

Estrutura de Pacotes

src/main/java/com/fiap/fase2/



Tecnologias Utilizadas

Tecnologia	Versão	Uso
Java	17	Linguagem principal
Spring Boot	3.2.3	Framework web
Spring Data JPA	-	Persistência
PostgreSQL	15	Banco de dados (produção)
H2 Database	-	Banco de dados (dev/test)
Docker & Compose	-	Containerização
Maven	3.8+	Build
SpringDoc OpenAPI	2.3.0	Documentação (Swagger)
JUnit 5 + Mockito	-	Testes
JaCoCo	0.8.11	Cobertura $\geq 80\%$
SonarCloud	-	Qualidade de código

Endpoints da API

Tipos de Usuário

Método	Endpoint	Descrição	Status
POST	/api/v1/user-types	Criar tipo	201, 400, 422
GET	/api/v1/user-types	Listar todos	200
GET	/api/v1/user-types/{id}	Buscar por ID	200, 404
PUT	/api/v1/user-types/{id}	Atualizar	200, 404, 422
DELETE	/api/v1/user-types/{id}	Remover	204, 404

Usuários

Método	Endpoint	Descrição	Status
POST	/api/v1/users	Criar usuário	201, 400, 422
GET	/api/v1/users	Listar todos	200
GET	/api/v1/users?name={nome}	Buscar por nome	200
GET	/api/v1/users/{id}	Buscar por ID	200, 404
PUT	/api/v1/users/{id}	Atualizar	200, 404
DELETE	/api/v1/users/{id}	Remover	204, 404
POST	/api/v1/users/login	Login	200, 422
PATCH	/api/v1/users/{id}/password	Trocar senha	200, 422, 404

Restaurantes

Método	Endpoint	Descrição	Status
POST	/api/v1/restaurants	Criar restaurante	201, 400, 422
GET	/api/v1/restaurants	Listar todos	200
GET	/api/v1/restaurants?ownerId={id}	Buscar por dono	200
GET	/api/v1/restaurants/{id}	Buscar por ID	200, 404
PUT	/api/v1/restaurants/{id}	Atualizar	200, 404, 422
DELETE	/api/v1/restaurants/{id}	Remover	204, 404, 422

Cardápio (MenuItem)

Método	Endpoint	Descrição	Status
POST	/api/v1/restaurants/{id}/menu-items	Criar item	201, 400, 404, 422
GET	/api/v1/restaurants/{id}/menu-items	Listar itens	200
GET	/api/v1/menu-items/{id}	Buscar por ID	200, 404
PUT	/api/v1/menu-items/{id}	Atualizar	200, 404, 422
DELETE	/api/v1/menu-items/{id}	Remover	204, 404, 422

Header X-User-Id

Os endpoints de Restaurant (POST, PUT, DELETE) e MenuItem (POST, PUT, DELETE) aceitam o header X-User-Id para identificar o usuário logado. Quando presente, valida que somente o proprietário pode alterar/deletar o restaurante e seus itens.

REGRAS DE NEGÓCIO

1. Tipo de usuário único: Não podem existir dois tipos com o mesmo nome.
2. Email e login únicos: Não podem existir dois usuários com mesmo email ou login.
3. Validação de senha: Mínimo 8 caracteres, pelo menos 1 maiúscula, 1 minúscula e 1 número.
4. Somente RESTAURANT_OWNER cria restaurante: Validação no CreateRestaurantUseCase.
5. Somente o proprietário altera/deleta: Validação via header X-User-Id.
6. Preço do item > 0: Validação no Create e Update de MenuItem.
7. Integridade referencial: Não é possível deletar tipo com usuários vinculados, nem restaurante com itens.

INSTRUÇÕES DE CONFIGURAÇÃO E EXECUÇÃO

Pré-requisitos

- Java 17+
- Maven 3.8+
- Docker e Docker Compose (ou Colima no macOS)

Execução via Docker Compose

`docker-compose up --build -d`

Acesse:

- Health: <http://localhost:8080/actuator/health>
- Swagger: <http://localhost:8080/swagger-ui.html>

Execução Local (dev com H2)

`mvn spring-boot:run -Dspring-boot.run.profiles=dev`

Script Runner

`./run.sh`

Menu interativo com opções para execução local, Docker, testes e limpeza de banco.

Testes

`mvn clean verify`

Cobertura $\geq 80\%$ validada pelo JaCoCo. Relatório em `target/site/jacoco/index.html`.

Banco de Dados (DBeaver)

Campo	Valor
Host	localhost
Port	5433
Database	fiap_fase2
Username	postgres
Password	postgres

TESTES E QUALIDADE

- 254 testes automatizados (unitários + integração + controller)
- Cobertura $\geq 80\%$ validada por JaCoCo e SonarCloud
- Quality Gate SonarCloud aprovado (0 bugs, 0 vulnerabilidades)
- Collection Postman com 128 requests e assertions automatizadas
- Evidência de testes gerada via Newman (HTML) ou shell puro (curl + jq)

DEPLOY

A aplicação está publicada no Render.com:

- URL: <https://one2adjt-fase-2.onrender.com>
- Swagger: <https://one2adjt-fase-2.onrender.com/swagger-ui.html>
- Deploy automático a cada push na branch main
- Banco: PostgreSQL provisionado no Render

DECISÕES TÉCNICAS

Decisão	Justificativa
Clean Architecture	Isolamento de regras de negócio, independência de framework
UUID como ID	Evita conflitos em sistemas distribuídos
PasswordHasher interface	Permite trocar implementação de criptografia sem alterar use cases
H2 em dev, PostgreSQL em prod	Agilidade no desenvolvimento, robustez em produção
Docker multi-stage build	Imagem menor e segura (só JRE no runtime)
ProblemDetail (RFC 7807)	Padrão internacional para respostas de erro
JaCoCo $\geq 80\%$	Garante qualidade mínima de testes
Header X-User-Id	Simula sessão/token sem implementar Spring Security